LAB 5 Report

Internet Edge + ISP Serial WAN + Microservices

CP352005 Network Programming | Khon Kaen University

0. สมาชิกกลุ่ม

ชื่อ-สกุล	รหัสนักศึกษา	Role
นายบุญปวีณ เรืองไพศาล	673380588-4	Network Architect, IaC/DevOps, QA/Test, Developer

1. Lab Objective

วัตถุประสงค์ของ Lab นี้คือการจำลองสถาปัตยกรรมเครือข่ายระดับองค์กร โดยใช้ Docker container เป็นอุปกรณ์เครือข่ายและเซิร์ฟเวอร์ เพื่อทดสอบการทำงานของ:

- Internet Edge Router (R1) + NAT Gateway
- ISP WAN Serial Link (100.10.10.0/30)
- Branch Router (R2)
- Microservices deployment บน ServerA และ ServerB
- Network Address Translation (NAT Overload / MASQUERADE)

Lab นี้ขยายจาก Lab 4 โดยเพิ่ม WAN topology และ Internet Edge architecture เพื่อจำลองระบบ distributed microservices ที่ทำงานข้ามเครือข่ายจริง

2. Network Topology

โครงสร้างเครือข่ายประกอบด้วย 4 subnets:

- internet_net : 10.255.0.0/24 — เชื่อมต่อ R1 กับ Internet
- isp_net : 100.10.10.0/30 — WAN ระหว่าง R1 และ R2
- lan_a_net : 192.168.10.0/24 — LAN A (ServerA, ClientA)
- lan_b_net : 192.168.20.0/24 — LAN B (ServerB, ClientB)

Address Table

Device	Interface	IP Address	Role
R1	eth0	10.255.0.2	Internet
R1	eth1	100.10.10.1	ISP WAN
R1	eth2	192.168.10.1	LAN A Gateway

R2	eth0	100.10.10.2	WAN
R2	eth1	192.168.20.1	LAN B Gateway
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B
ClientA	eth0	192.168.10.50	Test Client A
ClientB	eth0	192.168.20.50	Test Client B

3. Infrastructure Deployment

เริ่ม container infrastructure ด้วย Docker Compose:

```
$ docker compose down -v
$ docker compose up -d
```

ผลลัพธ์:

```
[+] Running 10/10
 Network networklab5_lan_b_net     Created
 Network networklab5_lan_a_net     Created
 Network networklab5_internet_net  Created
 Network networklab5_isp_net       Created
 Container ClientB                 Started
 Container R2                      Started
 Container ServerB                 Started
 Container ServerA                 Started
 Container ClientA                 Started
 Container R1                      Started
```

ตรวจสอบ containers ที่กำลังทำงาน:

```
$ docker ps
NAMES    IMAGE                STATUS
ClientB  python:3.10-alpine   Up
ServerB  python:3.10-alpine   Up
ServerA  python:3.10-alpine   Up
R2       alpine               Up
ClientA  python:3.10-alpine   Up
R1       alpine               Up
```

แสดงว่า network infrastructure ถูกสร้างสำเร็จ

4. Deploy Microservices

Microservices ถูก deploy บน ServerA และ ServerB ด้วย start_services.py:

```
$ docker exec -d ServerA sh -c "python /automation/start_services.py"
$ docker exec -d ServerB sh -c "python /automation/start_services.py"
```

ตรวจสอบ process:

```
$ docker exec ServerA ps
```

```
PID  USER  TIME  COMMAND
  1  root  0:00  tail -f /dev/null
 14  root  0:00  python /automation/start_services.py
 20  root  0:00  python /automation/upload_service.py
 21  root  0:00  python /automation/processing_service.py
 22  root  0:00  python /automation/ai_service.py
 23  root  0:00  python /automation/gateway_service.py
```

แสดงว่า microservice stack ทำงานแล้ว

5. Microservice Port Verification

ตรวจสอบ port ที่เปิดบน ServerA:

```
$ docker exec ServerA ss -tuln
tcp  LISTEN  0  5  0.0.0.0:8000  0.0.0.0:*
tcp  LISTEN  0  5  0.0.0.0:8001  0.0.0.0:*
tcp  LISTEN  0  5  0.0.0.0:8002  0.0.0.0:*
tcp  LISTEN  0  5  0.0.0.0:9000  0.0.0.0:*
```

Port	Service	Description
8000	Upload Service	รับไฟล์และเก็บลง mock_storage พร้อม metadata JSON
8001	Processing Service	ประมวลผลไฟล์ (thumbnail, resize, convert)
8002	AI Service	Mock AI วิเคราะห์ประเภทและ sentiment ของไฟล์
9000	Gateway Service	Orchestrator รวม pipeline Upload→Processing→AI

แสดงว่า microservices ถูก deploy ครบทุก service

6. Network Connectivity Test

ทดสอบ ping ข้าม LAN จาก ClientA (192.168.10.50) ไปยัง ServerB (192.168.20.10):

```
$ docker exec ClientA ping -c 3 192.168.20.10
PING 192.168.20.10: 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=62 time=0.365 ms
64 bytes from 192.168.20.10: seq=1 ttl=62 time=0.360 ms
64 bytes from 192.168.20.10: seq=2 ttl=62 time=0.262 ms
3 packets transmitted, 3 packets received, 0% packet loss
```

เส้นทาง packet: ClientA → R1 → ISP WAN → R2 → ServerB

แสดงว่า routing ระหว่าง LAN A และ LAN B ทำงานถูกต้อง (0% packet loss)

7. Cross-Site Microservice Test

7.1 Health Check — Upload Service

```
$ docker exec ClientA sh -c "curl -v http://192.168.20.10:8000/health"
```

```
* Established connection to 192.168.20.10 port 8000 from 192.168.10.50
< HTTP/1.0 200 OK
{"status": "ok", "service": "upload", "port": 8000}
```

7.2 Gateway Workflow — POST /process-file

```
$ docker exec ClientA sh -c "curl -X POST http://192.168.20.10:9000/process-file"
< HTTP/1.0 200 OK
{"status": "ok", "pipeline": [
  {"step": "upload",     "status": 200},
  {"step": "processing", "status": 200},
  {"step": "ai",         "status": 200, "confidence": 0.97}
]}
```

แสดงว่า ClientA สามารถเรียก microservices บน ServerB ผ่าน WAN ได้สำเร็จ

8. Internet Connectivity Test

ทดสอบ Internet access จาก ServerA ผ่าน NAT บน R1:

```
$ docker exec ServerA ping -c 3 8.8.8.8
64 bytes from 8.8.8.8: seq=0 ttl=62 time=22.923 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=15.247 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=33.680 ms
3 packets transmitted, 3 packets received, 0% packet loss
```

แสดงว่า ServerA สามารถออก Internet ผ่าน R1 (NAT) ได้

9. Router Verification

9.1 R1 Routing Table

```
$ docker exec R1 ip route
default via 10.255.0.1 dev eth0
10.255.0.0/24  dev eth0  proto kernel  src 10.255.0.2
100.10.10.0/30 dev eth1  proto kernel  src 100.10.10.1
192.168.10.0/24 dev eth2 proto kernel  src 192.168.10.1
192.168.20.0/24 via 100.10.10.2 dev eth1
```

9.2 R2 Routing Table

```
$ docker exec R2 ip route
default via 100.10.10.1 dev eth1
100.10.10.0/30  dev eth1  proto kernel  src 100.10.10.2
192.168.10.0/24 via 100.10.10.1 dev eth1
192.168.20.0/24 dev eth0 proto kernel  src 192.168.20.1
```

R1 default → Internet (10.255.0.1), R2 default → R1 (100.10.10.1)

10. NAT Verification

ตรวจสอบ NAT rule บน R1:

```
$ docker exec R1 iptables -t nat -L -n
Chain POSTROUTING (policy ACCEPT)
  MASQUERADE  all  --  0.0.0.0/0  0.0.0.0/0
```

Connection Tracking (ยืนยัน NAT translation จริง):

```
$ docker exec R1 cat /proc/net/nf_conntrack | grep 8.8.8.8
src=192.168.10.10 dst=8.8.8.8 => src=8.8.8.8 dst=10.255.0.2
```

พบ MASQUERADE rule แสดงว่า R1 ทำ NAT overload (PAT) สำหรับ LAN A และ LAN B

11. Microservice Code Overview

ระบบประกอบด้วย 5 ไฟล์หลักที่ทำงานร่วมกันเป็น pipeline:

11.1 start_services.py — Service Manager

จัดการการเริ่มและหยุด services ทั้งหมดด้วย ServiceManager class:

- เริ่ม Upload, Processing, AI และ Gateway service ตามลำดับด้วย subprocess.Popen
- ตรวจสอบ health แต่ละ service ผ่าน /health endpoint ก่อนเริ่ม service ถัดไป
- รัน quick integration test ผ่าน Gateway /process-file endpoint
- รองรับ Ctrl+C gracefully ด้วย signal handler

11.2 upload_service.py — Upload Service (Port 8000)

รับไฟล์จาก client และเก็บลง mock storage:

- POST /upload — รับ multipart file, validate ขนาด max 10 MB, บันทึกลง mock_storage/ พร้อม UUID
- GET /upload/{file_id} — ดึง metadata ของไฟล์ที่อัปโหลด
- DELETE /upload/{file_id} — ลบไฟล์และ metadata
- ใช้ aiofiles สำหรับ async file I/O, สร้าง JSON metadata ใน mock_metadata/

11.3 processing_service.py — Processing Service (Port 8001)

ประมวลผลไฟล์ด้วย operation ต่างๆ:

- POST /process/{file_id} — รัน thumbnail / resize / convert พร้อม simulated delay 2-5 วินาที
- POST /process/batch — ประมวลผลหลายไฟล์พร้อมกัน
- GET /process/{file_id}/status — ตรวจสอบสถานะการประมวลผล
- GET /process/operations — แสดง supported operations และ parameters

11.4 ai_service.py — AI Service (Port 8002)

วิเคราะห์ไฟล์ด้วย mock AI models:

- POST /analyze/{file_id} — วิเคราะห์ตาม analysis_type: general, vision, nlp, classification
- POST /analyze/batch — วิเคราะห์หลายไฟล์พร้อมกัน
- GET /analyze/{file_id}/history — ประวัติการวิเคราะห์
- GET /models — รายการ mock AI models ที่รองรับ (vision, nlp, classifier)

11.5 gateway_service.py — Gateway Service (Port 9000)

Orchestrator รวม pipeline Upload → Processing → AI:

- POST /process-file — รับไฟล์ รัน pipeline ครบ 3 ขั้นตอน คืน WorkflowResponse
- POST /upload-only — อัปโหลดโดยไม่ประมวลผล
- POST /process-existing/{file_id} — ประมวลผลไฟล์ที่มีอยู่แล้ว
- GET /health — ตรวจสอบ health ของทุก service พร้อมกัน
- GET /stats — สถิติ gateway

12. Architectural Significance

Concept	Implementation
Enterprise Edge Router	R1 (NAT + ISP WAN)
ISP Serial WAN	100.10.10.0/30
Branch Router	R2
Service Distribution	ServerA / ServerB
NAT Boundary	R1 (MASQUERADE)
Cross-Site Services	Gateway API :9000

13. Conclusion

Lab 5 ประสบความสำเร็จในการสร้าง distributed microservice architecture ที่ทำงานข้าม WAN network ผลลัพธ์สำคัญ:

- Routing ระหว่าง LAN A และ LAN B ทำงานถูกต้อง (0% packet loss)
- Cross-site microservice access ผ่าน WAN สำเร็จ (HTTP 200 OK)
- Internet connectivity ผ่าน NAT MASQUERADE ทำงาน
- Microservices stack (Upload / Processing / AI / Gateway) ทำงานครบทุก service

Lab นี้แสดงให้เห็นการทำงานของ Internet Edge Routing, WAN connectivity, Network segmentation และ Distributed service architecture ซึ่งเป็นพื้นฐานของระบบ cloud และ enterprise networking